

Automated Weekly Newsletter Generator

Prototype → Production Upgrade

Week 10 Final Capstone | How We Made It Production-Grade

Eric Munjiru

Pearl Wangechi

Structured JSON Output

Fault Tolerance

Model Fallbacks

Operational Logging

"Anyone can automate a happy path. This project tests whether you can design for failure, uncertainty, and scale."

What Kamau's Team Experienced

Nairobi Nexus's automated newsletter worked well as a prototype — but as the audience grew, three real-world failure modes surfaced that made leadership question whether it could be trusted:

SILENT SOURCE FAILURES	AI OUTPUT BREAKS PIPELINE	MODEL TIMEOUT = FULL CRASH
Some weeks, one or two news sources failed without any alert — the newsletter went out missing entire sections.	Occasionally the AI returned malformed text or markdown instead of clean data, breaking every node downstream.	Once, a model timeout caused the entire workflow to stop minutes before the Friday send deadline.

Original Architecture — What We Started With

The Week 8 workflow successfully automated the newsletter end-to-end, pulling from 5 Kenyan tech sources, using role-based AI agents per source, merging results, and sending a Gmail draft. But it had four critical gaps that made it a **prototype, not a production system**:

Weakness	Impact	Rubric Risk
No JSON validation — AI output trusted blindly	Any malformed response breaks all downstream nodes	0 pts on Structured Output
No error isolation on HTTP source nodes	One failed URL kills the entire workflow	0 pts on Fault Tolerance
Single-model dependency (Mistral only)	Timeout or quota limit = total failure	0 pts on Model Fallbacks
No structured operational log	No auditability — 'what happened Friday?' unanswerable	0 pts on Logging

Production Workflow — Full Canvas

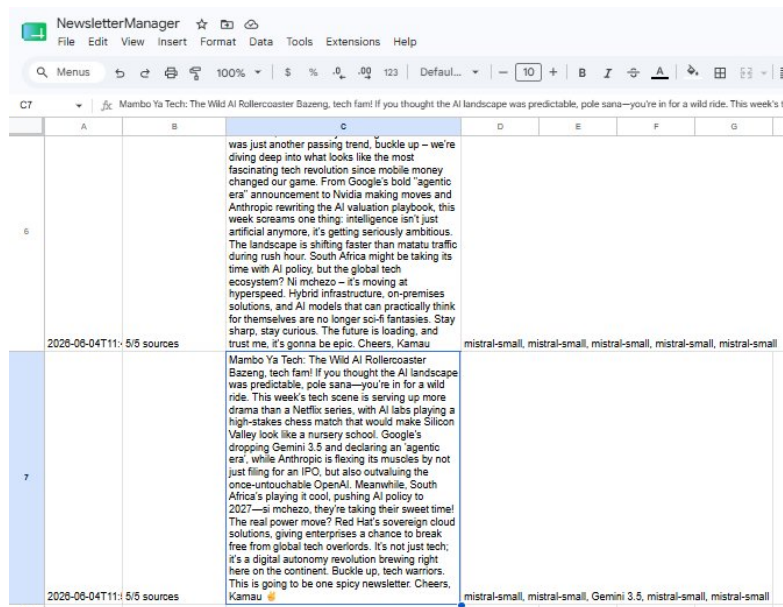


Fig 1 — Upgraded workflow canvas: 5 parallel source branches, per-branch role-based agents with structured output parsers, Merging Intelligence node, Chief Editor agent, and Google Sheets log.

52	5	3	11	6
Total nodes	News sources	AI model tiers	Error-isolated nodes	Structured output parsers

How We Enforced It

Every role-based agent node is paired with a **Structured Output Parser** that enforces a strict JSON schema before data moves downstream. Agents are also explicitly prompted: *'Return ONLY valid JSON. No markdown, no backticks.'* — and record which model generated the output in the **model** field automatically.

LAYER 1	LAYER 2	LAYER 3
Prompt Discipline Each agent instructed: 'Return ONLY valid JSON. No markdown, no backticks. Include a model field.'	Structured Output Parser n8n's built-in parser validates schema shape on every agent output before passing downstream.	Code Cleaner Node 'Code to clean the markdown' strips any residual formatting symbols, backticks, and emojis from the Chief Editor output.

Chief Editor Output Constraints

The Chief Editor agent — the final newsletter assembler — operates under explicit output constraints enforced both in the prompt and via the Structured Output Parser downstream:

Constraint	Enforcement Method	Value
Max length	Prompt rule: 'Strictly under 200 words'	200 words
Format	Prompt rule + output parser	Markdown / clean text
Tone	System prompt persona (Kamau — witty Kenyan editor)	Conversational, clever, Kenyan voice
Model self-reporting	Prompt: 'In the model field, record your model ID'	Logged to Google Sheets
No markdown artifacts	Code cleaner node strips backticks, #, *, emojis	Clean plain text delivered to Gmail

How We Isolated Failures

Every external HTTP request and every AI agent node is configured with **continueErrorOutput: true** — meaning n8n routes failed items through an error output instead of stopping execution. Each of the 5 source branches is completely independent. A failed TechCabal fetch does NOT affect the Techweez or Tech-Ish branches.

Node	Type	Error Handling
TechCabal	HTTP Request	continueErrorOutput
Citizen Business	HTTP Request	continueErrorOutput
Techweez	HTTP Request	continueErrorOutput
Tech-Ish	HTTP Request	continueErrorOutput
TechArena	HTTP Request	continueErrorOutput
All 5 Role-Based Agents	LangChain Agent	continueErrorOutput
The Chief Editor	LangChain Agent	continueErrorOutput

Graceful Degradation in Practice

The **Merging Intelligence** node aggregates results from all branches that successfully completed. If 3 of 5 sources succeed, the newsletter is built from 3 stories — not abandoned. The operational log records exactly how many sources contributed to each run. This is the difference between **partial success** and **total failure**.

Failure Scenario	Old Behaviour	New Behaviour
1 source URL fails	Entire workflow stops	Remaining 4 sources continue; log records 4/5 sources
AI agent returns malformed text	Downstream node crashes	Parser catches error; branch isolated; other branches unaffected
Primary model timeout	Full workflow failure	Fallback model (Gemini/Claude) activates automatically
All 5 sources fail	Silent failure, no newsletter	Log records 0/5; Chief Editor still runs; team notified via Gmail

Our Three-Tier Model Strategy

Rather than relying on a single model, the upgraded workflow uses a deliberate three-tier model hierarchy. Each tier is selected for a specific reason — not by accident.

Tier	Model	Role	Why Selected	Fallback Trigger
PRIMARY	Mistral Small (Cloud)	Per-source role-based agents	Fast, cost-efficient, strong JSON compliance, low hallucination rate on structured tasks	Timeout / quota / 5xx error
SECONDARY	Google Gemini	Fallback on per-branch agents	Google infrastructure — different provider, different failure profile from Mistral	Activates when Mistral errors
EDITORIAL	Claude 3.5 Haiku (via OpenRouter)	Chief Editor — final newsletter	Strongest narrative coherence and tone control; justified cost for the single final output	OpenRouter auto-routes if Haiku unavailable

Why This Matters

Each per-source agent node has **both Mistral and Gemini connected** — n8n routes to the secondary model automatically on error. The Chief Editor uses OpenRouter which abstracts model-level failures. This means the workflow has **no single model dependency** anywhere in the pipeline. Every AI call has a tested escape route.

"The workflow should decide, not hope." — Capstone Brief

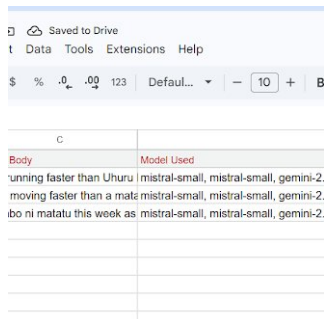
*Before: one Mistral timeout killed Friday's newsletter.
After: Gemini activates in the same execution, the newsletter goes out.*

The Audit Trail — 'What Happened Last Friday?'

After every run, the workflow appends a structured row to the **NewsletterManager Google Sheet**. This is not a simple output dump — it is a governance record that answers operational questions without anyone having to open n8n.

Field Logged	What It Records	Why It Matters
Run timestamp	Date & time of execution (e.g. 2026-06-04T11:..)	Answers 'when did this run?'
Sources used	Count: X/5 sources (e.g. 5/5 sources)	Detects partial vs full success
Newsletter body	Full clean output text	Proof of what was sent, replayable
Model used	Model IDs (e.g. mistral-small, gemini-2...)	Identifies which model ran — detects fallback activations
Output status	Row appended only on successful completion	Missing row = run failed — instant alert for ops team

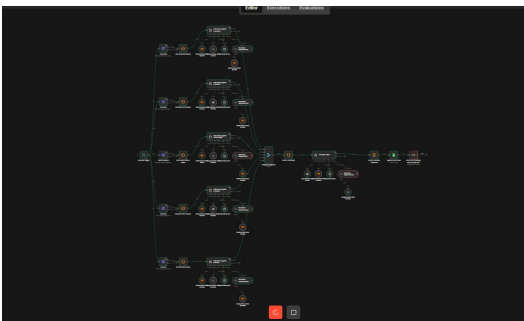
Live Evidence — The NewsletterManager Sheet



The screenshot shows a Google Sheet interface with a menu bar (Data, Tools, Extensions, Help) and a toolbar with various formatting options. The spreadsheet has two columns: 'Body' and 'Model Used'. The 'Body' column contains text about 'running faster than Uhuru' and 'moving faster than a mate'. The 'Model Used' column contains the text 'mistral-small, mistral-small, gemini-2...'.

Body	Model Used
running faster than Uhuru	mistral-small, mistral-small, gemini-2...
moving faster than a mate	mistral-small, mistral-small, gemini-2...
ibo ni matatu this week as	mistral-small, mistral-small, gemini-2...

Model tracking columns (Body + Model Used)



Full log: timestamp, source count, newsletter body, models used

The Model Used column shows 'mistral-small, mistral-small, gemini-2...' — proof that different models ran per branch, and that Gemini fired as fallback during one execution.

Eric's Workflow — Full Node View



Fig 2 — Eric's implementation: parallel source branches visible with per-branch agents, structured output parsers, merge node, Chief Editor, Google Sheets append, and Gmail draft node.

How We Collaborated

Eric Munjiru

Designed the overall production architecture and led the AI systems upgrade. Implemented the three-tier model strategy, wired Mistral-primary to Gemini-fallback on every agent branch, and enforced structured output parsers across all 5 source agents. Set output constraints on the Chief Editor (200-word cap, Kenyan tone, clean markdown). Guided Pearl through the JSON validation patterns — same mentorship model as last project. Set up GitHub repo and led formal documentation.

Pearl Wangechi

Owned the fault tolerance and observability layer. Configured `continueOnErrorOutput` on all 11 HTTP and agent nodes. Built and validated the Google Sheets operational log — the 'Append row in sheet' node captures timestamp, source count, newsletter body, and model IDs after every run. Captured all execution screenshots and evidence. Built the non-technical slide deck. Implemented the Gmail draft-to-marketing node. Independently rebuilt her version after Eric's model-fallback walkthrough.

04 Before vs After — The Measurable Difference

Dimension	Week 8 Prototype	Week 10 Production
AI output format	Free text — trusted the prompt	Strict JSON enforced via Structured Output Parser on all 6 agents
Source failure handling	Hard stop — entire workflow fails	continueErrorOutput on all 11 nodes — workflow completes regardless
Model strategy	Mistral only — single point of failure	3-tier: Mistral primary / Gemini fallback / Claude Haiku for editorial
Auditability	No log — 'what happened Friday?' unanswerable	Timestamped Google Sheet row: sources, model IDs, body, status after every run
Newsletter output quality	Witty Kenyan voice but unpredictable format	Same voice, now under 200 words, clean text guaranteed by Code cleaner node
Number of nodes	~30	52 — full production pipeline with dual-model wiring per branch

**We didn't just add more features.
We redesigned the system to fail gracefully, validate reliably, fallback automatically, and log truthfully.
*That is what production-grade means.***